```
#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone {
  Inner,
  Middle,
  Outer
};

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

// Calculate fare using switch and integer arithmetic
long long fare(Zone z, bool peak) {
  long long f = 0;

  switch (z) {
    case Zone::Inner:
      f = BASE_PAISE[0];
      break;

    case Zone::Middle:
      f = BASE_PAISE[1];
      break;

    case Zone::Outer:
      f = BASE_PAISE[2];
      break;
```

```
        default:
            return -1;
    }

    if (peak) {
        f = f * (100 + PEAK_SURCHARGE_PERCENT) / 100;
    }

    return f;
}

// Check minimum remaining balance
bool canTravel(long long balance, long long f) {
    return f >= 0 &&
           balance >= f &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000; // Rs. 100.00

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
```

```
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {
            balance -= f;

            cout << "Fare Rs "
                 << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ')
                 << " balance Rs " << balance / 100 << '\n';
        } else {
            cout << "Denied (low balance)";
            cout << " balance Rs " << balance / 100 << '\n';
        }
    }

    return 0;
}
```